



Московский государственный университет имени М.В.Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра суперкомпьютеров и квантовой информатики

Агаджанян Грант Минасович

**Методы предсказания производительности
инференса моделей трансформеров
определённого класса**

Выпускная квалификационная работа

Научный руководитель:
к.ф.-м.н. Попова Нина Николаевна

Москва, 2026

Аннотация

Методы предсказания производительности инференса моделей трансформеров определённого класса

Агаджанян Грант Минасович

Выпускная квалификационная работа посвящена разработке и исследованию методов предсказания производительности инференса моделей трансформеров определённого класса, а именно BERT-подобных энкодерных моделей, на фиксированной вычислительной платформе GPU NVIDIA A100. Рассматривается задача предсказания трёх целевых метрик инференса по описанию архитектуры модели и параметрам запуска: среднего времени инференса на один батч, пикового использования GPU-памяти и энергопотребления. Актуальность работы обусловлена широким применением BERT-подобных моделей, разнообразием их архитектурных конфигураций и высокой стоимостью прямого перебора большого числа параметров запуска инференса на целевой платформе.

В работе выполнен обзор существующих подходов к предсказанию метрик инференса нейросетевых моделей, включая аналитические методы, методы, основанные на профилировании, и методы, основанные на машинном обучении. Предложен метод, основанный на совместном использовании структурного представления модели в виде вычислительного графа и агрегированных глобальных признаков, описывающих архитектуру модели и конфигурацию запуска. Для узлов вычислительного графа сформировано локальное признаковое описание, учитывающее тип операции, её параметры и масштабные характеристики, а для модели в целом — набор глобальных признаков, отражающих архитектурные параметры и оценку вычислительной сложности.

Для решения поставленной задачи разработаны программные средства, обеспечивающие автоматическое построение вычислительного графа, извлечение локальных и глобальных признаков, измерение целевых метрик инференса и формирование обучающей выборки. Обучающая выборка построена на основе гото-

вых моделей из библиотеки Hugging Face и пользовательских BERT-конфигураций, полученных варьированием ключевых архитектурных параметров. Для каждой пары «модель — конфигурация запуска» формируется экземпляр, включающий граф вычислений, признаки и измеренные значения целевых метрик. Итоговая выборка содержит порядка 10^4 экземпляров.

На построенной выборке исследованы несколько классов предсказательных моделей: линейные модели, ансамблевые методы на основе деревьев решений, градиентный бустинг, полносвязные нейронные сети и графовая нейронная сеть. Экспериментальное исследование показало, что наилучшее качество по основной метрике времени инференса обеспечивает модель HistGradientBoosting. В режиме переноса на новые модели (*group_model*) наилучший результат достигается при использовании представления *pooled*, а графовая нейронная сеть демонстрирует практически сопоставимое качество, что подтверждает информативность графового представления вычислений. В режиме экстраполяции на наиболее крупные архитектуры (*largest20*) наиболее устойчивой оказывается модель HistGradientBoosting на *static*-признаках. Показано также, что наиболее сложной для предсказания целевой метрикой является пиковое использование GPU-памяти.

Практическая значимость работы состоит в том, что после этапа оффлайн-сбора обучающей выборки на целевой вычислительной платформе обученный предиктор может использоваться без постоянного доступа к ней и выступать в качестве её программного суррогата для рассматриваемого класса моделей. Предложенный подход может быть использован для предварительной оценки затрат на инференс и для поддержки выбора эффективных конфигураций запуска BERT-подобных моделей.

Содержание

1	Введение	6
2	Постановка задачи	12
3	Обзор проблематики по теме работы	15
3.1	Общая постановка задачи предсказания метрик инференса	15
3.2	Аналитические модели	15
3.3	Методы, основанные на профилировании	17
3.4	Модели на основе машинного обучения	18
3.4.1	Табличные модели	19
3.4.2	Графовые нейронные сети и модель DIPPM	19
3.5	Сравнение подходов	20
3.6	Специфика BERT-подобных моделей	20
3.7	Выводы из обзора	22
4	Исследование и построение решения задачи	25
4.1	Общая схема предложенного метода	25
4.2	Структурное представление модели	26
4.2.1	Построение вычислительного графа	26
4.2.2	Классификация операций	26
4.3	Формирование признакового пространства	28
4.3.1	Локальные признаки узлов графа	28
4.3.2	Глобальные признаки модели и конфигурации запуска	31
4.4	Построение обучающей выборки	31
4.4.1	Выбор набора моделей и конфигураций запуска	33
4.4.2	Измерение целевых метрик инференса на фиксированной вычис- лительной платформе	34
4.4.3	Формирование одного экземпляра и полной обучающей выборки .	35
4.5	Предсказательные модели $\mathcal{F}_\theta$	35
4.5.1	Неграфовые модели и используемые представления	36
4.5.2	Графовая нейронная сеть	36
4.6	Особенности обучения и критерии качества	37

4.7	Вычислительная платформа	39
5	Программная реализация метода	40
5.1	Общая архитектура программной реализации	40
5.2	Модуль формирования набора моделей	41
5.3	Модуль построения обучающей выборки	41
5.3.1	Построение графового представления	43
5.3.2	Измерение целевых метрик	44
5.3.3	Организация хранения выборки	44
5.4	Модули обучения предсказательных моделей	45
5.4.1	Модуль обучения неграфовых моделей	45
5.4.2	Модуль обучения графовой модели	46
5.5	Модуль разбиения данных и оценки качества	46
5.6	Используемые программные библиотеки	47
5.7	Особенности реализации	47
6	Результаты экспериментов	50
6.1	Результаты в режиме <i>group_model</i>	50
6.2	Результаты в режиме <i>largest20</i>	51
6.3	Сравнение режимов и представлений	52
6.4	Анализ вычислительной стоимости применения предиктора	53
6.5	Выводы по результатам экспериментов	55
	Заключение	57
	Список литературы	60

1 Введение

За последние годы модели глубокого обучения стали одной из ключевых технологий в широком спектре прикладных задач: от машинного перевода и диалоговых систем до рекомендательных сервисов, анализа изображений и мультимодальной обработки данных. Особое место среди современных архитектур занимают модели семейства Transformer, предложенные в работе “Attention Is All You Need” [1]. Благодаря механизму многоголового внимания (multi-head attention, МНА) и отказу от рекуррентных вычислений трансформеры обеспечили существенный прогресс в задачах обработки естественного языка, а затем были успешно адаптированы и к другим предметным областям.

Одновременно с ростом качества моделей увеличиваются их размеры и вычислительная сложность. Современные трансформерные архитектуры могут содержать сотни миллионов и миллиарды параметров, а их инференс требует значительных вычислительных ресурсов, прежде всего графических ускорителей. В прикладных сценариях использования моделей, таких как онлайн-сервисы, рекомендательные системы и интеллектуальные ассистенты, важнейшими характеристиками становятся время отклика, использование видеопамати и энергопотребление. Избыточный запас по вычислительным ресурсам приводит к росту стоимости эксплуатации, тогда как недостаточный запас может вызывать деградацию качества сервиса, снижение пропускной способности и отказы из-за нехватки памяти.

На практике основные метрики инференса конкретной модели на целевой вычислительной платформе обычно определяются эмпирически: модель запускается при заданных параметрах инференса, после чего измеряются время выполнения, пиковое использование GPU-памяти и энергопотребление. При рассмотрении большого числа архитектурных вариантов и конфигураций запуска стоимость такого перебора может существенно возрастать. Это особенно важно в задачах выбора допустимых и эффективных параметров инференса. В связи с этим возникает задача построения предсказательной модели, которая для конкретной модели и заданных параметров запуска позволяет быстро и достаточно точно оценивать основные метрики инференса без необходимости выполнять полный набор прямых запусков на GPU.

Существующие подходы к предсказанию производительности инференса можно условно разделить на три основные группы.

К первой группе относятся *аналитические методы*, основанные на оценке числа

арифметических операций, сложности отдельных слоёв и характеристик вычислительного устройства. Такие методы, как правило, обладают хорошей интерпретируемостью и невысокой вычислительной стоимостью, однако они не в полной мере учитывают накладные расходы программных фреймворков, особенности реализации операторов и влияние реальной среды исполнения [2, 3, 4].

Вторую группу составляют *методы, основанные на профилировании*, в которых сначала измеряются характеристики отдельных операторов или блоков, а затем метрики всей модели оцениваются на основе композиции этих измерений. Подобные методы обычно обеспечивают более высокую точность, однако требуют доступа к целевой аппаратной платформе и ограниченно учитывают эффекты взаимодействия операторов, перераспределения памяти и глобальную структуру вычислительного графа модели [5, 6, 7].

К третьей группе относятся *методы, основанные на машинном обучении*. В этих подходах строится отображение из признакового описания модели и параметров запуска в интересующие метрики производительности. В качестве предсказательных моделей используются, в частности, многослойные персептроны, модели на основе деревьев решений, градиентный бустинг и графовые нейронные сети, работающие с представлением нейронной сети в виде графа вычислений [8, 9, 10]. Следует отметить, что в рассматриваемой supervised-постановке такие методы требуют предварительного формирования обучающей выборки, состоящей из признаковых описаний моделей и конфигураций запуска, а также соответствующих измеренных значений целевых метрик инференса. Эти целевые значения — время выполнения, использование памяти и энергопотребление — должны быть получены с помощью прямых запусков на целевой аппаратной платформе, поэтому доступ к ней необходим на этапе сбора обучающих данных. После обучения предсказательная модель может применяться для оценки новых конфигураций без выполнения полного набора прямых измерений и без постоянного доступа к целевой платформе.

Среди современных работ данного направления особый интерес представляет модель DIPPM (Deep Learning Inference Performance Predictive Model) [11], в которой архитектура нейронной сети представляется в виде графа, а для предсказания времени инференса, использования памяти и энергопотребления применяется графовая нейронная сеть. Результаты DIPPM показывают, что графовое представление архитектуры

является перспективным инструментом для задач прогнозирования производительности. При этом исходная работа ориентирована преимущественно на свёрточные модели компьютерного зрения и не учитывает в явном виде особенности трансформерных архитектур.

Трансформеры, в частности энкодерные модели, обладают рядом структурных особенностей, существенно влияющих на характеристики инференса. К ним относятся механизм многоголового внимания, остаточные связи, операции нормализации, полносвязные преобразования в блоках внимания и feed-forward-блоках, а также зависимость вычислительной стоимости от длины входной последовательности токенов и размера батча.

Семейство трансформерных моделей неоднородно. В нём можно выделить, в частности, encoder-only модели, decoder-only модели и encoder-decoder архитектуры. Общая схема исходной архитектуры Transformer приведена на рис. 1.

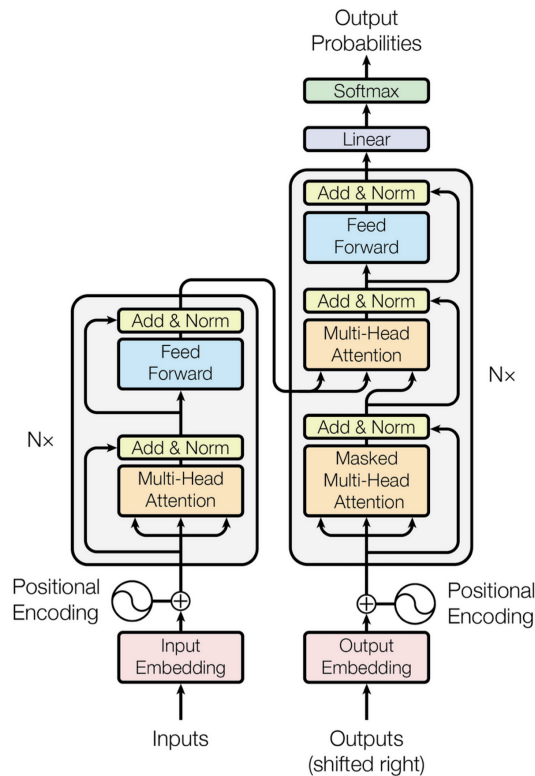


Рис. 1: Общая архитектура модели Transformer: слева показан стек энкодерных блоков, справа — стек декодерных блоков [1].

В настоящей работе рассматриваются BERT-подобные модели, то есть трансформеры энкодерного типа, использующие только энкодерную часть общей схемы. Такие модели строятся как стек энкодерных блоков с механизмом многоголового самовнимания, полносвязными feed-forward-блоками, остаточными связями и слоями нормализации. Они не выполняют авторегрессионное декодирование, а обрабатывают входную последовательность целиком, что отличает их от decoder-only моделей, используемых, например, в генеративных сценариях.

Выбор BERT-подобных моделей в качестве объекта исследования обусловлен несколькими причинами. Во-первых, они широко применяются в прикладных задачах обработки текста, включая классификацию, ранжирование, поиск и анализ текстовых данных. Во-вторых, этот класс моделей является архитектурно достаточно однородным: большинство рассматриваемых моделей имеют общую схему из повторяющихся энкодерных блоков, но различаются числом слоёв, скрытой размерностью, числом голов внимания, размером промежуточного слоя FFN и параметрами запуска инференса. Это позволяет систематически исследовать влияние архитектурных характеристик и конфигурации запуска на метрики инференса.

Актуальность настоящей работы определяется следующими обстоятельствами:

- широким распространением BERT-подобных трансформерных моделей в прикладных и промышленных системах;
- разнообразием архитектурных и конфигурационных вариантов BERT-подобных моделей;
- необходимостью выбора допустимых и эффективных конфигураций запуска инференса на целевой вычислительной платформе;
- ресурсоёмкостью прямого перебора большого числа конфигураций запуска, требующего многократных измерений на GPU;
- практической значимостью одновременного прогнозирования нескольких метрик производительности: времени инференса, пикового использования GPU-памяти и энергопотребления.

Целью работы является разработка и исследование методов оценки и повышения эффективности инференса BERT-подобных трансформерных моделей на основе пред-

сказания производительности инференса по архитектуре модели и параметрам запуска на целевой вычислительной платформе. Под производительностью в работе понимается совокупность трёх метрик: среднего времени инференса на один батч, пикового использования GPU-памяти и энергопотребления.

Для достижения поставленной цели в работе решаются следующие **задачи**:

1. проанализировать существующие подходы к предсказанию метрик инференса нейросетевых моделей;
2. уточнить рассматриваемый класс BERT-подобных энкодерных моделей и набор параметров запуска, используемых при предсказании метрик инференса;
3. разработать способ представления BERT-подобной модели и конфигурации запуска в виде признакового описания, включающего структурную информацию о вычислительном графе;
4. построить обучающую выборку измерений производительности инференса на целевой вычислительной платформе;
5. разработать и программно реализовать метод предсказания метрик инференса по архитектуре модели и параметрам запуска;
6. обучить на построенной обучающей выборке несколько классов предсказательных моделей, включая классические модели машинного обучения, полносвязные нейросети и графовые нейронные сети;
7. провести экспериментальное исследование качества предсказания времени инференса, пикового использования GPU-памяти и энергопотребления.

Объектом исследования являются BERT-подобные энкодерные трансформерные модели. Предметом исследования являются методы предсказания метрик производительности их инференса на целевой вычислительной платформе.

Основная идея предлагаемого подхода состоит в том, чтобы представить BERT-подобную модель как граф операций, где вершины соответствуют операциям вычислительного графа, а рёбра отражают зависимости между ними. Для такого графа формируется признаковое описание, включающее, с одной стороны, локальные характеристики узлов, а с другой стороны — глобальные признаки, описывающие архитектуру

модели и параметры запуска. На основе полученного представления обучаются предсказательные модели, оценивающие интересующие метрики производительности.

Практическая значимость работы состоит в том, что предложенный подход может быть использован для предварительной оценки затрат на инференс BERT-подобных трансформерных моделей, сокращения числа дорогостоящих экспериментальных запусков на GPU и поддержки выбора допустимых конфигураций запуска с точки зрения времени, памяти и энергопотребления. После оффлайн-сбора обучающей выборки на целевой платформе обученная предсказательная модель может использоваться как её программный суррогат для рассматриваемого класса моделей и конфигураций запуска.

2 Постановка задачи

Рассматривается задача предсказания основных метрик инференса BERT-подобных моделей на фиксированной вычислительной платформе.

Дано:

- $\mathcal{M}$ — BERT-подобная энкодерная трансформер-модель;
- $c = (B, L)$ — конфигурация запуска инференса, где B — размер батча, L — максимальная длина входной последовательности токенов;
- D — фиксированная рабочая нагрузка, то есть набор входных данных, на котором выполняется инференс;
- $\mathcal{G}$ — фиксированное целевое GPU.

Для каждой пары $(\mathcal{M}, c)$ при фиксированных D и $\mathcal{G}$ определён вектор метрик инференса

$$\mathbf{y}(\mathcal{M}, c) = (T(\mathcal{M}, c), M_{\text{peak}}(\mathcal{M}, c), E(\mathcal{M}, c)),$$

где

- $T(\mathcal{M}, c)$ — среднее время инференса на один батч;
- $M_{\text{peak}}(\mathcal{M}, c)$ — пиковое использование GPU-памяти во время инференса;
- $E(\mathcal{M}, c)$ — энергия, затраченная GPU при выполнении инференса.

Пусть $\mathbf{x}(\mathcal{M}, c)$ — признаковое описание модели $\mathcal{M}$ и конфигурации запуска c . В данной работе оно строится на основе структурного представления модели в виде графа вычислений и набора глобальных признаков, описывающих архитектуру модели и параметры запуска инференса.

Требуется построить предсказательную модель

$$\mathcal{F}_{\theta} : \mathbf{x}(\mathcal{M}, c) \mapsto \hat{\mathbf{y}}(\mathcal{M}, c),$$

где

$$\hat{\mathbf{y}}(\mathcal{M}, c) = (\hat{T}(\mathcal{M}, c), \hat{M}_{\text{peak}}(\mathcal{M}, c), \hat{E}(\mathcal{M}, c))$$

— предсказанный вектор метрик инференса, а

$$\theta = (\theta_1, \theta_2, \dots, \theta_k), \quad \theta_i \in \mathbb{R},$$

— обучаемые параметры модели $\mathcal{F}_\theta$, причём k — число параметров предсказательной модели, определяемое её архитектурой.

Для настройки обучающих параметров предсказательной модели предполагается наличие обучающей выборки, полученной по результатам прямых измерений на целевой вычислительной платформе:

$$\mathcal{S} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N,$$

где

$$\mathbf{x}_i = \mathbf{x}(\mathcal{M}_i, c_i), \quad \mathbf{y}_i = \mathbf{y}(\mathcal{M}_i, c_i).$$

Иными словами, каждый элемент обучающей выборки соответствует одной паре “модель, конфигурация запуска” и содержит её признаковое описание, а также измеренные значения целевых метрик инференса.

В общем виде обучение модели $\mathcal{F}_\theta$ формулируется как задача минимизации эмпирического риска:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta),$$

где

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \ell(\hat{\mathbf{y}}_i, \mathbf{y}_i), \quad \hat{\mathbf{y}}_i = \mathcal{F}_\theta(\mathbf{x}_i),$$

а $\ell(\cdot, \cdot)$ — функция потерь, задающая ошибку между предсказанным и истинным векторами метрик. Конкретный вид функции потерь, используемый при обучении моделей, описывается далее в разделе, посвящённом построению решения.

Качество предсказательной модели оценивается по средней абсолютной процентной ошибке отдельно для каждой целевой компоненты:

$$\begin{aligned} \text{MAPE}_T &= \frac{100\%}{N} \sum_{i=1}^N \left| \frac{T_i - \hat{T}_i}{T_i} \right|, \\ \text{MAPE}_M &= \frac{100\%}{N} \sum_{i=1}^N \left| \frac{M_{\text{peak},i} - \hat{M}_{\text{peak},i}}{M_{\text{peak},i}} \right|, \\ \text{MAPE}_E &= \frac{100\%}{N} \sum_{i=1}^N \left| \frac{E_i - \hat{E}_i}{E_i} \right|. \end{aligned}$$

Таким образом, задача работы состоит в построении модели $\mathcal{F}_\theta$, которая по описанию архитектуры BERT-подобной модели и параметрам запуска инференса позволяет предсказывать среднее время инференса, пиковое использование GPU-памяти и энергопотребление на фиксированном GPU $\mathcal{G}$.

3 Обзор проблематики по теме работы

В данном разделе рассматриваются существующие подходы к предсказанию метрик инференса нейросетевых моделей, анализируются их преимущества и ограничения, а также обсуждается применимость этих подходов к архитектурам класса Transformer и, в частности, к BERT-подобным энкодерным моделям.

3.1 Общая постановка задачи предсказания метрик инференса

Задача предсказания метрик инференса нейросетевых моделей возникает в ряде практически значимых сценариев. К их числу относятся:

- подбор вычислительной платформы для заданной модели;
- анализ зависимости метрик инференса от параметров запуска;
- автоматизированный поиск и отбор архитектурных конфигураций модели;
- выбор допустимых и эффективных конфигураций запуска инференса;
- планирование вычислительной нагрузки в серверных системах и центрах обработки данных.

Во всех перечисленных случаях представляет интерес модель вида

$$f(\text{нейросеть, параметры запуска, вычислительная платформа}) \longrightarrow (\text{время, память, энергия}),$$

которая позволяла бы заранее получать оценки основных метрик инференса без полного перебора всех рассматриваемых конфигураций прямыми запусками. Как было отмечено во введении, существующие подходы к решению этой задачи условно делятся на три основные группы: аналитические методы, методы, основанные на профилировании, и методы, основанные на машинном обучении.

3.2 Аналитические модели

Аналитические модели строятся на основе априорных представлений о вычислительной сложности операций и характеристиках аппаратной платформы. Как правило, в таких подходах:

1. оценивается количество арифметических операций, объём обращений к памяти и иные агрегированные характеристики вычислений;
2. задаётся упрощённая модель устройства, учитывающая, например, пиковую производительность, пропускную способность памяти и объём доступной GPU-памяти;
3. на основе этих величин вычисляются оценки интересующих метрик инференса.

Подобные идеи реализованы, в частности, в работах Paleo, DNNMem, LLMem [2, 3, 4]. В Paleo предлагается аналитическая модель производительности глубоких нейронных сетей, позволяющая оценивать время выполнения и масштабируемость систем глубокого обучения по описанию архитектуры модели, аппаратной платформы, сетевой среды и программного стека. Такой подход удобен для быстрой предварительной оценки различных системных конфигураций и анализа масштабируемости. В DNNMem рассматривается задача аналитической оценки потребления GPU-памяти: метод систематически учитывает память, связанную с вычислительным графом модели, промежуточными тензорами и runtime-расходами фреймворка. В LLMem аналогичная идея развивается применительно к крупным языковым моделям: оценивается пиковое потребление GPU-памяти при различных режимах распределённого fine-tuning и выбирается допустимая схема запуска.

Сильная сторона аналитических моделей состоит в том, что они не требуют предварительного обучения на большом наборе измерений и позволяют быстро получать первичные оценки временных и ресурсных затрат. Кроме того, такие методы, как правило, не требуют выполнения прямых измерений на целевой вычислительной платформе, хотя могут использовать её априорно заданные характеристики, такие как пиковая производительность, пропускная способность памяти и объём доступной памяти. Они полезны для грубой фильтрации заведомо недопустимых конфигураций, оценки порядка вычислительной сложности и предварительного планирования экспериментов.

Вместе с тем такие модели имеют ряд принципиальных ограничений:

- **Ограниченный учёт особенностей программной среды:** реальные накладные расходы фреймворков, запуск вычислительных ядер, синхронизация, работа механизма выделения памяти и другие runtime-эффекты учитываются приближённо;

- **Недостаточный учёт структуры вычислительного графа:** агрегированные характеристики не всегда отражают топологию вычислений, порядок выполнения операций и зависимости между промежуточными тензорами;
- **Приближённый учёт параметров запуска.** Конфигурация запуска инференса часто учитывается лишь на уровне изменения числа операций, тогда как реальные характеристики инференса существенно зависят от особенностей исполнения на GPU: эффективности использования параллелизма, пропускной способности памяти, поведения кешей и оптимизаций вычислительных ядер. В результате аналитические модели не всегда способны точно воспроизводить зависимость производительности инференса от параметров запуска.

3.3 Методы, основанные на профилировании

Методы второй группы опираются на прямые измерения характеристик выполнения отдельных операторов, вычислительных ядер или типовых блоков модели. Общая идея состоит в том, что поведение полной модели аппроксимируется на основе профиля её компонентов.

Обычно:

- предварительно профилируются ключевые операторы, вычислительные ядра или типовые блоки модели;
- строится база измерений для целевой платформы или семейства устройств;
- метрики полной модели оцениваются как композиция измеренных характеристик с учётом структуры модели и порядка выполнения операций.

Характерным примером такого подхода является nn-Meter [6]. В этой работе инференс модели представляется не просто как последовательность исходных операторов нейронной сети, а как набор вычислительных kernel-блоков, то есть единиц выполнения после оптимизаций runtime-системы и фьюзинга операторов. Ключевая идея состоит в автоматическом выделении таких kernel-блоков и построении предикторов задержки для каждого из них. Затем задержка полной модели оценивается на основе предсказаний по отдельным kernel-блокам. Такой подход позволяет лучше учитывать особен-

ности реального исполнения на edge-устройствах, то есть на вычислительных устройствах, расположенных на периферии сети и непосредственно выполняющих инференс, по сравнению с простым суммированием стоимостей отдельных слоёв.

Другой пример профилировочного направления связан с runtime-ориентированными предсказателями, такими как Habitat [7]. В подобных работах используется информация о реальном выполнении модели или её компонентов, чтобы предсказывать стоимость обучения или инференса в различных системных условиях. Преимущество таких методов состоит в том, что они учитывают особенности конкретного программно-аппаратного стека более точно, чем чисто аналитические модели.

Главным преимуществом методов профилирования является более точный учёт реального поведения системы. За счёт использования измерений на целевой платформе такие методы в большей степени отражают особенности конкретной версии фреймворка, драйвера, библиотеки вычислительных примитивов и runtime-оптимизаций.

В то же время им присущи следующие ограничения:

- требуется доступ к целевой платформе для построения профиля;
- необходим значительный объём предварительных измерений, особенно если рассматривается много операторов, размеров тензоров и режимов запуска;
- ограниченно учитываются глобальные взаимодействия между операторами, включая перераспределение памяти и эффекты всего вычислительного графа;
- при смене аппаратной платформы, версии фреймворка или библиотеки вычислительных примитивов может потребоваться повторное профилирование.

3.4 Модели на основе машинного обучения

Третья группа подходов использует методы машинного обучения для построения отображения из признакового описания модели и параметров запуска в метрики инференса. В качестве предсказательных моделей могут использоваться:

- классические ML-модели, включая линейные модели, ансамбли деревьев решений и градиентный бустинг;
- полносвязные нейронные сети;

- графовые нейронные сети;

В supervised-постановке требуется обучающая выборка, сформированная на основе прямых измерений на целевой платформе. Каждый элемент такой выборки содержит признаковое описание модели и конфигурации запуска, а также измеренные значения целевых метрик. Однако после обучения предсказательная модель позволяет оценивать новые конфигурации без выполнения полного набора прямых запусков и без постоянного доступа к целевой платформе.

3.4.1 Табличные модели

В табличных ML-подходах модель описывается набором агрегированных признаков: числом слоёв, количеством параметров, FLOPs или MAC-операций, размерами входных данных, характеристиками устройства и параметрами запуска. На основе такого описания может обучаться линейная модель, многослойный персептрон, случайный лес, Extra Trees или градиентный бустинг.

Преимущество табличных моделей состоит в простоте реализации, компактности признакового пространства и высокой скорости работы после обучения. Такие методы хорошо подходят в тех случаях, когда зависимость между архитектурными параметрами и метриками инференса в достаточной степени отражается агрегированными характеристиками.

Основное ограничение табличного подхода состоит в том, что агрегированное описание теряет часть информации о структуре вычислительного графа. Две модели с близким числом параметров или близкой оценкой числа операций могут иметь различную организацию вычислений, отличающийся порядок операций и разную структуру промежуточных тензоров. Для трансформеров это особенно существенно, поскольку производительность определяется не только масштабом модели, но и структурой блоков самовнимания, FFN-блоков, нормализаций и остаточных связей.

3.4.2 Графовые нейронные сети и модель DIPPM

Следующим этапом развития ML-подходов стало использование структурного представления нейросетевой модели в виде вычислительного графа. Одной из наиболее близких к настоящей работе является модель DIPPM [11].

В статье DIPPM предлагается рассматривать нейросетевую модель как граф

$$(X, Q),$$

где X — матрица признаков узлов вычислительного графа, а Q — матрица смежности, описывающая зависимости между операциями. Узлы графа соответствуют операциям модели, а рёбра отражают передачу данных между ними. На основе такого представления обучается графовая нейронная сеть, формирующая эмбединг графа и предсказывающая метрики инференса: время выполнения, использование памяти и энергопотребление.

Основное достоинство такого подхода состоит в том, что он учитывает не только агрегированные характеристики модели, но и топологию вычислений. Это особенно важно для предсказания производительности, поскольку порядок выполнения операций и структура зависимостей между ними могут существенно влиять на время инференса, использование памяти и энергопотребление.

Вместе с тем исходная постановка DIPPM ориентирована преимущественно на свёрточные нейронные сети и общий набор моделей компьютерного зрения. Поэтому её прямое применение к BERT-подобным трансформерам требует адаптации признакового описания: необходимо явно учитывать операции самовнимания, линейные преобразования в блоках внимания, FFN-блоки, LayerNorm, а также зависимость вычислительной стоимости от размера батча и длины входной последовательности токенов.

3.5 Сравнение подходов

В таблице 1 приведено обобщённое сравнение рассмотренных групп подходов.

3.6 Специфика BERT-подобных моделей

Для BERT-подобных энкодерных моделей характерна достаточно регулярная структура: они состоят из повторяющихся энкодерных блоков, включающих механизм многоголового самовнимания, FFN-блоки, операции нормализации и остаточные связи. Эта регулярность делает возможным построение единого признакового описания для широкого набора архитектур данного класса.

Таблица 1: Сравнение подходов к предсказанию метрик инференса

Подход	Основная идея	Преимущества	Ограничения
Аналитические модели	Оценка метрик на основе числа операций, объёма памяти и модели аппаратной платформы	Интерпретируемость; низкая вычислительная стоимость; не требуется обучающая выборка	Упрощённый учёт runtime-эффектов, структуры графа и параметров запуска
Методы профилирования	Измерение операторов, kernel-блоков или типовых фрагментов модели с последующей композицией оценок	Более точный учёт реального поведения платформы и программного стека	Требуется доступ к платформе; большой объём предварительных измерений; необходимость повторного профилирования
Табличные ML-модели	Обучение регрессионной модели по агрегированным признакам архитектуры и конфигурации запуска	Высокая скорость приращения предиктора; простота реализации; хорошо работают при информативных глобальных признаках	Требуется обучающая выборка; теряется часть информации о структуре вычислительного графа
Графовые ML-модели	Представление модели как графа операций и обучение GNN для предсказания метрик	Учитывают топологию вычислений; потенциально лучше отражают структуру модели	Более сложная реализация; требуется построение графа и обучающая выборка; качество зависит от признаков узлов и полноты данных

В то же время метрики инференса BERT-подобных моделей зависят не только от агрегированных характеристик, таких как число параметров или оценка числа операций. Существенную роль играют:

- операции самовнимания, стоимость которых зависит от длины входной последовательности токенов;
- линейные преобразования в блоках внимания и FFN-блоках;
- операции LayerNorm, softmax, reshape и поэлементные операции;
- размер батча и максимальная длина последовательности токенов;
- структура вычислительного графа и порядок выполнения операций.

Следовательно, для данного класса моделей требуется специализированное признаковое описание, учитывающее как глобальные архитектурные параметры, так и локальную структуру вычислительного графа.

Несмотря на большое число работ по оценке производительности инференса нейросетевых моделей, публикаций, специально ориентированных именно на BERT-подобные энкодерные трансформеры, сравнительно немного. Чаще рассматриваются либо общие предсказательные модели для широкого класса нейросетевых архитектур, либо специализированные фреймворки для decoder-only LLM, такие как NeuSight [12], LLMCompass [13], ReaLLM [14], RAPID-LLM [15] и SweetSpot [16]. В таблице 2 приведены наиболее близкие к настоящей работе публикации, в которых рассматривается предсказание характеристик инференса трансформеров и близких архитектур.

3.7 Выводы из обзора

Проведённый анализ показывает:

1. аналитические модели позволяют быстро получать приближённые оценки, но недостаточно полно учитывают runtime-эффекты и структуру вычислительного графа;
2. методы, основанные на профилировании, лучше отражают реальное поведение платформы, но требуют существенных предварительных измерений;

Таблица 2: Работы по предсказанию метрик инференса для трансформеров и близких архитектур

Работа	Класс моделей	Метрики	Подход	Сообщаемый результат
NeuSight (2024/2025)	BERT-Large, GPT2-Large, GPT-3 и др.	задержка инференса	аппаратно-ориентированный предиктор на основе машинного обучения.	для режима fused inference: ошибка 18.9% для BERT-Large, 12.5% для GPT2-Large и 15.7% в среднем по fused-моделям
DIPPM (2023)	общие модели глубокого обучения	задержка, память, энергия	GNN по обобщённому вычислительному графу	MAPE до 1.9% в задаче предсказания метрик инференса на NVIDIA A100
LLMCompass (2023/2024)	LLM декодерного типа, в том числе GPT-3 175B	задержка инференса (prefill/decode)	аналитико-симуляционная система для оценки аппаратного исполнения	средняя ошибка 4.1% для LLM; отдельно 0.69% для prefill и 7.5% для decode
ReaLLM (2025)	LLaMA-70B и другие сценарии использования LLM	сквозная задержка инференса	трассо-ориентированное имитационное моделирование с библиотекой профилей вычислительных ядер	средняя ошибка предсказания задержки 9.1% на задачах инференса на 4×H100
RAPID-LLM (2025)	инференс моделей семейства Llama, нагрузки масштаба GPT	задержка шага инференса	аппаратно-ориентированная модель операторного уровня и системная симуляция	ошибка предсказания задержки одного шага инференса в пределах 10.4% на проверочных сценариях для A100
SweetSpot (2026)	LLM 1B–9B (OPT, LLaMA, Gemma, Falcon, Qwen2, Granite)	потребление энергии	аналитическая модель, учитывающая вычислительную сложность и сложность обращений к памяти	среднее значение MAPE 1.79% при моделировании кривой энергоэффективности и энергопотребления на H100

3. методы машинного обучения после обучения позволяют быстро оценивать новые конфигурации, однако требуют обучающей выборки и качественного признакового описания;
4. графовое представление является перспективным способом учёта структуры вычислений, но для BERT-подобных моделей оно требует адаптации к операциям самовнимания, FFN-блокам, нормализациям и зависимости от параметров запуска.

Таким образом, задача разработки и исследования метода предсказания метрик инференса для BERT-подобных моделей остаётся актуальной. Она требует учёта как структуры вычислительного графа, так и параметров запуска инференса, а также экспериментального сравнения различных классов предсказательных моделей.

4 Исследование и построение решения задачи

В данном разделе описывается предлагаемое решение задачи предсказания метрик инференса BERT-подобных энкодерных трансформер-моделей. Основная идея метода состоит в том, чтобы представить рассматриваемую модель в виде вычислительного графа операций и на его основе сформировать признаковое описание, включающее локальные характеристики узлов графа, а также глобальные признаки архитектуры модели и параметров запуска инференса.

4.1 Общая схема предложенного метода

Предлагаемое решение основано на двух взаимодополняющих способах описания модели и конфигурации запуска инференса:

- структурное представление модели в виде вычислительного графа прямого прохода, отражающего состав операций и зависимости между ними;
- агрегированное представление, включающее глобальные характеристики архитектуры модели и конфигурации запуска.

При этом агрегированное описание не противопоставляется вычислительному графу как независимый источник информации. Часть глобальных признаков задаётся непосредственно архитектурными параметрами модели и параметрами запуска инференса, а часть вычисляется на основе построенного графа операций. Таким образом, вычислительный граф в предлагаемом методе служит не только самостоятельным структурным представлением модели, но и источником признаков для последующего агрегирования.

Общая схема метода включает следующие этапы:

1. формирование набора BERT-подобных моделей $\mathcal{M}$, обеспечивающего разнообразие по основным архитектурным параметрам, и набора конфигураций запуска инференса c , покрывающего различные значения размера батча B и длины входной последовательности токенов L ;
2. для каждой пары $(\mathcal{M}, c)$:
 - (a) построение вычислительного графа модели;

- (b) классификация операций графа с выделением типов операций, характерных для трансформеров;
 - (c) формирование локальных и глобальных признаков;
 - (d) измерение целевых метрик инференса на фиксированной вычислительной платформе $\mathcal{G}$;
3. формирование обучающей выборки;
 4. построение и обучение на сформированной выборке нескольких классов моделей $\mathcal{F}_\theta$, предсказывающих указанные метрики инференса.

4.2 Структурное представление модели

4.2.1 Построение вычислительного графа

Для каждой рассматриваемой модели $\mathcal{M}$ строится вычислительный граф прямого прохода. Вершины графа соответствуют операциям, выполняемым при инференсе, а рёбра отражают зависимости по данным между ними. Такое представление позволяет перейти от описания модели на уровне крупных архитектурных блоков к уровню отдельных операций.

Использование вычислительного графа в задаче предсказания метрик инференса обусловлено тем, что реальное поведение модели определяется не только её агрегированными характеристиками, такими как число параметров и оценка вычислительной сложности, но и структурой зависимостей между операциями, порядком выполнения вычислений и размерами промежуточных тензоров.

4.2.2 Классификация операций

В работе используется специализированная классификация операций, ориентированная на BERT-подобные энкодерные трансформеры. В вычислительном графе выделяются 16 типов узлов, приведённых в таблице 3.

Такая таксономия позволяет явно выделить операции механизма внимания, FFN-блоков, нормализации, поэлементных преобразований и изменения формы тензоров. Именно эти группы операций в значительной степени определяют вычислительную стоимость инференса BERT-подобных моделей.

Таблица 3: Типы узлов вычислительного графа и их описание

Тип узла	Описание
<code>embedding</code>	Операции эмбеддингов, формирующие начальные представления входных токенов.
<code>linear_q</code>	Линейное преобразование, формирующее запросы (query) в механизме самовнимания.
<code>linear_k</code>	Линейное преобразование, формирующее ключи (key) в механизме самовнимания.
<code>linear_v</code>	Линейное преобразование, формирующее значения (value) в механизме самовнимания.
<code>linear_attn_out</code>	Выходное линейное преобразование после объединения голов механизма внимания.
<code>linear_ffn_in</code>	Первое линейное преобразование в FFN-блоке, переводящее представление в промежуточное пространство.
<code>linear_ffn_out</code>	Второе линейное преобразование в FFN-блоке, возвращающее представление в пространство скрытого состояния.
<code>linear_other</code>	Прочие линейные преобразования, не отнесённые к специализированным категориям внимания и FFN.
<code>layernorm</code>	Операции нормализации типа LayerNorm, а также родственные им операции нормализации.
<code>attention_matmul</code>	Операции вычисления механизма внимания, включая матричные умножения при вычислении весов внимания и применении этих весов к значениям, а также fused-реализации attention.
<code>softmax</code>	Операции softmax и log_softmax, используемые для нормализации весов внимания.
<code>gelu</code>	Нелинейные функции активации, применяемые в модели, включая GELU и другие близкие активации.
<code>dropout</code>	Операции Dropout, используемые для регуляризации.
<code>elementwise</code>	Поэлементные, логические, маскирующие и редукционные операции, не выделенные в отдельные специализированные классы.
<code>reshape</code>	Операции изменения формы, переупорядочивания, разбиения, объединения и индексирования тензоров, включая reshape, view, permute, transpose и родственные преобразования.
<code>other</code>	Прочие операции, не попавшие в перечисленные выше категории.

4.3 Формирование признакового пространства

4.3.1 Локальные признаки узлов графа

Каждому узлу вычислительного графа сопоставляется вектор локальных признаков фиксированной размерности 28. Он состоит из трёх частей:

1. one-hot кодировки типа операции размерности 16;
2. набора атрибутивных признаков размерности 8;
3. набора масштабных признаков размерности 4.

Атрибутивные признаки описывают параметры конкретной операции и её положение в структуре модели. Используемые в работе атрибутивные признаки приведены в таблице 4.

Третий блок локальных признаков образуют четыре масштабных коэффициента

$$(k_B, k_L, k_H, k_I),$$

характеризующие ожидаемую вычислительную стоимость операции от основных параметров масштаба задачи. Здесь k_B соответствует зависимости от размера батча B , k_L — от длины входной последовательности токенов L , k_H — от размера скрытого состояния H , а k_I — от размера промежуточного слоя FFN I .

Значения коэффициентов (k_B, k_L, k_H, k_I) не извлекаются непосредственно из вычислительного графа, а задаются эвристически в зависимости от типа операции и её контекста в модели. Используемые в работе значения приведены в таблице 5.

Использование масштабных коэффициентов обусловлено тем, что время выполнения операции и объём требуемых ресурсов определяются не только её типом и параметрами, но и характером роста вычислительной стоимости при изменении конфигурации запуска и архитектурных параметров модели. Поэтому включение этих признаков в локальное описание узла делает представление вычислительного графа более информативным с точки зрения предсказания метрик инференса.

Таким образом, локальные признаки описывают не только тип операции и её параметры, но и положение операции в структуре модели, а также характер зависимости её вычислительной стоимости от параметров масштаба задачи.

Таблица 4: Атрибутивные признаки узлов вычислительного графа

Признак	Описание
<code>in_dim</code>	Оценка размерности входа операции. Для линейных преобразований определяется по размерности матрицы весов и соответствует числу входных признаков. Для остальных операций, если точное значение не извлекается, может принимать значение по умолчанию.
<code>out_dim</code>	Оценка размерности выхода операции. Для линейных преобразований определяется по размерности матрицы весов и соответствует числу выходных признаков; в остальных случаях, если возможно, берётся последняя размерность выходного тензора.
<code>param_count</code>	Число обучаемых параметров, связанных с данной операцией, выраженное в тысячах параметров.
<code>num_heads</code>	Число голов внимания для операций, отнесённых к механизму самовнимания. Для остальных операций принимает значение 0.
<code>intermediate_size</code>	Размер промежуточного слоя FFN для операций, отнесённых к полносвязной части энкодерного блока. Для остальных операций принимает значение 0.
<code>layer_idx_norm</code>	Нормализованный индекс энкодерного слоя, в котором расположена операция: $\frac{\text{layer_idx}}{\max(\text{num_layers}-1, 1)}$. Для операций вне энкодерных слоёв принимает значение -1 .
<code>batch_size</code>	Размер батча, используемый при данной конфигурации запуска инференса.
<code>seq_len</code>	Максимальная длина входной последовательности токенов для данной конфигурации запуска инференса.

Таблица 5: Эвристическое задание масштабных признаков локального описания узла

Тип узла	(k_B, k_L, k_H, k_I)	Интерпретация
<code>attention_matmul</code>	$(1, 2, 1, 0)$	Квадратичная зависимость от длины последовательности токенов, характерная для операций механизма самовнимания, и линейная от размера батча.
<code>softmax</code> в механизме внимания	$(1, 2, 0, 0)$	Линейная зависимость от размера батча и квадратичная от длины последовательности токенов при нормализации матрицы весов внимания.
<code>linear_q</code> , <code>linear_k</code> , <code>linear_v</code> , <code>linear_attn_out</code> , <code>linear_other</code>	$(1, 1, 2, 0)$	Линейные преобразования с линейной зависимостью от батча, длины последовательности и квадратичной зависимостью от скрытой размерности.
<code>linear_ffn_in</code> , <code>linear_ffn_out</code>	$(1, 1, 1, 1)$	Линейные преобразования FFN, линейно зависящие от батча, длины последовательности токенов, скрытой размерности и промежуточной размерности FFN.
<code>embedding</code>	$(1, 1, 1, 0)$	Линейная зависимость от батча, длины последовательности и скрытой размерности.
<code>layernorm</code> , <code>gelu</code> , <code>dropout</code> , <code>elementwise</code>	$(1, 1, 1, 0)$	Операции, вычислительная стоимость которых линейно зависит от числа обрабатываемых элементов представления.
<code>reshape</code>	$(0, 0, 0, 0)$	Операции изменения формы тензоров, для которых масштабная вычислительная стоимость в признаковом описании не учитывается.
<code>other</code>	зависит от контекста	Для прочих операций значения задаются по контексту: $(1, 2, 1, 0)$ для <code>attention</code> -связанных операций и $(1, 1, 1, 0)$ в остальных случаях.

4.3.2 Глобальные признаки модели и конфигурации запуска

Наряду с локальными признаками формируется вектор глобальных признаков $F_s \in \mathbb{R}^{16}$, компоненты которого приведены в таблице 6. Эти признаки описывают модель и конфигурацию запуска инференса на агрегированном уровне и дополняют информацию, содержащуюся в вычислительном графе. Часть глобальных признаков задаётся непосредственно архитектурными параметрами модели и параметрами запуска, а часть вычисляется на основе статистик построенного вычислительного графа.

Признак `total_macs_linear` задаёт оценку суммарного числа MAC-операций во всех линейных преобразованиях модели. В реализации он вычисляется на основе явного обхода линейных узлов вычислительного графа: для каждой линейной операции суммируется произведение числа элементов её выходного тензора на входную размерность соответствующего преобразования. Полученная на трассировочной конфигурации величина нормируется на число токенов и затем масштабируется на текущую конфигурацию запуска.

Признак `approx_attn_macs` представляет собой аналитическую оценку вычислительной стоимости блоков самовнимания и FFN. В используемой реализации она вычисляется по формуле

$$\text{approx_attn_macs} = \text{num_layers} * (4BLH^2 + 2BL^2H + 2BLHI),$$

где B — размер батча, L — длина последовательности, H — размер скрытого состояния, I — размер промежуточного слоя FFN, а `num_layers` — число энкодерных слоёв.

Совместное использование локальных и глобальных признаков позволяет описывать модель одновременно на уровне структуры вычислений и на уровне агрегированных характеристик архитектуры и конфигурации запуска.

4.4 Построение обучающей выборки

Для формирования обучающей выборки рассматривается множество BERT-подобных энкодерных трансформеров, включающее как готовые модели из библиотеки Hugging Face, так и BERT-конфигурации, полученные путём варьирования ключевых архитектурных параметров. Для каждой выбранной модели задаётся набор конфигураций запуска инференса, определяемых размером батча B и максимальной длиной входной

Таблица 6: Глобальные признаки модели и конфигурации запуска

Признак	Описание
<code>batch_size</code>	Размер батча B , используемый при инференсе.
<code>seq_len</code>	Максимальная длина входной последовательности токенов L .
<code>batch_size × seq_len</code>	Произведение $B \cdot L$, характеризующее общий масштаб входной нагрузки на модель.
<code>batch_size × seq_len²</code>	Величина $B \cdot L^2$, отражающая квадратичную по длине последовательности составляющую вычислительной стоимости механизма самовнимания.
<code>num_layers</code>	Число энкодерных слоёв модели.
<code>hidden_size</code>	Размер скрытого состояния модели.
<code>num_heads</code>	Число голов внимания.
<code>intermediate_size</code>	Размер промежуточного слоя FFN-блока.
<code>total_macs_linear</code>	Оценка суммарного числа MAC-операций во всех линейных преобразованиях модели. В реализации вычисляется на трассировочной конфигурации по линейным узлам вычислительного графа, нормируется на число токенов и затем масштабируется на текущую конфигурацию запуска.
<code>approx_attn_macs</code>	Аналитическая оценка вычислительной стоимости блоков самовнимания и FFN, вычисляемая по архитектурным параметрам модели и конфигурации запуска.
<code>total_params</code>	Общее число обучаемых параметров модели, выраженное в миллионах параметров.
<code>frac_linear</code>	Доля узлов вычислительного графа, отнесённых к линейным преобразованиям.
<code>frac_attn</code>	Доля узлов вычислительного графа, отнесённых к классу <code>attention_matmul</code> .
<code>frac_ln</code>	Доля узлов вычислительного графа, отнесённых к операциям нормализации.
<code>frac_sm</code>	Доля узлов вычислительного графа, отнесённых к классу <code>softmax</code> .
<code>frac_do</code>	Доля узлов вычислительного графа, отнесённых к классу <code>dropout</code> .

последовательности токенов L . Такое построение выборки позволяет охватить как разнообразие архитектур рассматриваемого класса моделей, так и различные режимы их использования при инференсе.

4.4.1 Выбор набора моделей и конфигураций запуска

Качество предсказательной модели существенно зависит от разнообразия обучающей выборки. Поэтому при её построении необходимо охватить как широкий спектр архитектур BERT-подобных моделей, так и различные режимы их использования при инференсе.

Обучающая выборка формировалась на основе двух источников:

- готовых BERT-подобных моделей из библиотеки Hugging Face;
- BERT-конфигураций, построенных с помощью сетки архитектурных параметров.

В набор готовых моделей были включены архитектуры из различных семейств BERT-подобных энкодеров, основные характеристики которых приведены в таблице 7.

Таблица 7: Готовые BERT-подобные модели из Hugging Face, использованные при построении обучающей выборки

Модель	Семейство	Слои	H	num_heads	I
albert-base-v2	ALBERT	12	768	12	3072
bert-base-uncased	BERT	12	768	12	3072
distilbert-base-uncased	DistilBERT	6	768	12	3072
google/electra-small-discrimin.	ELECTRA	12	256	4	1024
google/mobilebert-uncased	MobileBERT	24	512	4	512
microsoft/deberta-v3-small	DeBERTa-v3	6	768	12	3072
microsoft/deberta-v3-xsmall	DeBERTa-v3	12	384	6	1536
prajjwal1/bert-tiny	Compact BERT	2	128	2	512
prajjwal1/bert-mini	Compact BERT	4	256	4	1024
prajjwal1/bert-small	Compact BERT	4	512	8	2048
prajjwal1/bert-medium	Compact BERT	8	512	8	2048
roberta-base	RoBERTa	12	768	12	3072
roberta-large	RoBERTa	24	1024	16	4096

Дополнительно использовались BERT-конфигурации, формируемые путём варьирования ключевых параметров архитектуры:

- числа энкодерных слоёв L ;
- размера скрытого состояния H ;
- числа голов внимания `num_heads`;
- размера промежуточного слоя FFN I .

Используемые значения параметров:

$$L \in \{2, 4, 6, 8, 12, 16, 24\},$$

$$H \in \{128, 192, 256, 384, 512, 768, 1024\},$$

$$\text{num_heads} \in \{1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 48, 64\},$$

$$I \in \{256, 384, 512, 576, 768, 1024, 1152, 1536, 2048, 2304, 3072, 4096, 4608, 6144\}.$$

Для каждой выбранной модели рассматривается набор конфигураций запуска инференса, задаваемых парой

$$c = (B, L).$$

В работе использовался перебор значений этих параметров по степеням двойки:

$$B \in \{1, 2, 4, 8, 16, 32\}, \quad L \in \{1, 2, 4, 8, 16, 32, 64, 128\}.$$

Такой выбор конфигураций запуска позволяет исследовать поведение моделей в широком диапазоне режимов инференса — от малых до сравнительно более ресурсоёмких.

4.4.2 Измерение целевых метрик инференса на фиксированной вычислительной платформе

Для каждой пары “модель $\mathcal{M}$, конфигурация запуска c ” на фиксированной вычислительной платформе $\mathcal{G}$ измеряются целевые метрики инференса

$$\mathbf{y} = (T, M_{\text{peak}}, E),$$

где T — среднее время инференса на один батч, M_{peak} — пиковое использование GPU-памяти, а E — энергия, затраченная GPU при выполнении инференса. Эти величины используются в качестве целевых переменных в рамках задачи обучения с учителем.

4.4.3 Формирование одного экземпляра и полной обучающей выборки

Для каждой пары “модель $\mathcal{M}$, конфигурация запуска c ” формируется один экземпляр обучающей выборки вида

$$(W_i, X_i, F_{s,i}, T_i, M_i, E_i),$$

где W_i — список рёбер вычислительного графа, X_i — матрица локальных признаков узлов, $F_{s,i}$ — вектор глобальных признаков, а T_i, M_i, E_i — измеренные значения целевых метрик.

Таким образом, каждый экземпляр обучающей выборки объединяет три уровня информации:

- структурное представление модели в виде графа вычислений;
- локальные и глобальные признаки, описывающие архитектуру модели и конфигурацию запуска;
- целевые метрики инференса, измеренные на фиксированной вычислительной платформе.

Полная обучающая выборка формируется как совокупность таких экземпляров для всех рассматриваемых моделей и конфигураций запуска. Итоговая обучающая выборка включает около 10^4 экземпляров.

4.5 Предсказательные модели $\mathcal{F}_\theta$

В работе рассматриваются несколько классов предсказательных моделей:

- простая базовая модель `Dummy`;
- линейные модели `Linear`, `Ridge` и `ElasticNet`;
- ансамблевые модели на основе деревьев решений `Random Forest` и `Extra Trees`;
- модель градиентного бустинга `HistGradientBoosting`;
- полносвязная нейронная сеть `MLP`;
- графовая нейронная сеть `GNN`.

4.5.1 Неграфовые модели и используемые представления

Для неграфовых предсказательных моделей используются два варианта признакового описания:

- *static* — только глобальные признаки модели и конфигурации запуска;
- *pooled* — глобальные признаки, дополненные агрегированными статистиками по локальным признакам узлов вычислительного графа.

В варианте *pooled* по матрице локальных признаков X вычисляются агрегаты по узлам, в частности среднее значение, стандартное отклонение и максимум. Это позволяет использовать информацию о вычислительном графе в табличных моделях без явного применения графовой нейронной сети.

Преимущество неграфовых моделей состоит в простоте применения и малой стоимости предсказания после обучения. При этом их качество существенно зависит от того, насколько информативным является выбранное табличное представление.

4.5.2 Графовая нейронная сеть

Для графовой модели используется полное графовое представление вычислений. Входом служат:

- список рёбер графа W ;
- матрица признаков узлов X ;
- вектор глобальных признаков F_s .

Архитектура используемой графовой модели представлена на рисунке 2. Графовая часть модели основана на нескольких последовательно применяемых слоях GraphSAGE, каждый из которых сопровождается нелинейностью ReLU и слоем Dropout. На выходе графовой части формируются эмбединги узлов, которые затем агрегируются с помощью операции глобального усреднения (Global Mean Pool), в результате чего получается единый эмбединг всего вычислительного графа.

Далее полученный эмбединг графа объединяется с вектором глобальных признаков F_s и подаётся в полносвязную регрессионную часть модели. Полносвязная часть состоит

из нескольких последовательно применяемых блоков “полносвязный слой — ReLU — Dropout” и формирует итоговый вектор предсказаний.

Выходной слой модели задаёт три предсказанные метрики инференса:

$$\hat{\mathbf{y}} = (\hat{T}, \hat{M}_{\text{peak}}, \hat{E}),$$

где $\hat{T}$ — предсказанное среднее время инференса на один батч, $\hat{M}_{\text{peak}}$ — предсказанное пиковое использование GPU-памяти, а $\hat{E}$ — предсказанная энергия, затраченная GPU при выполнении инференса.

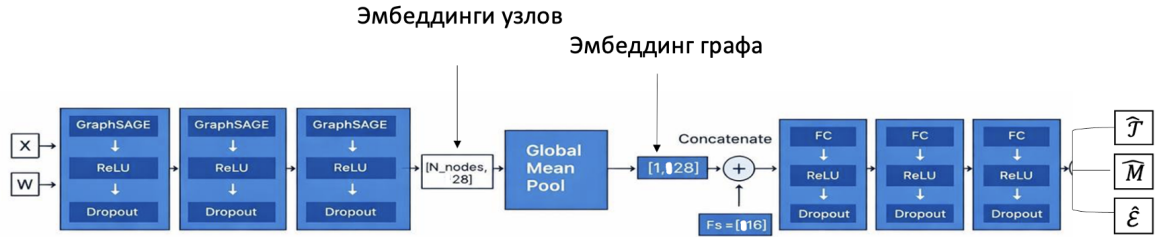


Рис. 2: Архитектура предложенной графовой модели

4.6 Особенности обучения и критерии качества

Обучение предсказательных моделей проводится в многокритериальной постановке, поскольку требуется одновременно предсказывать три целевые метрики инференса:

$$\mathbf{y}_i = (T_i, M_{\text{peak},i}, E_i), \quad \hat{\mathbf{y}}_i = (\hat{T}_i, \hat{M}_{\text{peak},i}, \hat{E}_i),$$

где $\mathbf{y}_i$ — вектор метрик инференса для i -го экземпляра обучающей выборки, а $\hat{\mathbf{y}}_i = \mathcal{F}_\theta(\mathbf{x}_i)$ — соответствующий ему предсказанный вектор.

Для подбора гиперпараметров предсказательных моделей используется механизм GridSearchCV, позволяющий выполнять систематический перебор заранее заданных наборов гиперпараметров с оценкой качества моделей на кросс-валидации. Это обеспечивает единообразный подход к настройке различных моделей и позволяет выбирать конфигурации, обеспечивающие наилучшее качество предсказания на валидационных данных.

При проведении вычислительных экспериментов рассматриваются два основных режима обучения и оценки качества предсказательных моделей. В режиме `group_model` проверяется способность предсказательной модели к переносу на конфигурации моделей, полностью отсутствующие в обучающей выборке: одна и та же архитектурная конфигурация целиком попадает только в одно из подмножеств `train`, `val` или `test`. В режиме `largest20` оценивается способность модели к экстраполяции на более крупные архитектуры: в тестовую выборку выделяются около 20% крупнейших конфигураций моделей, тогда как обучение и валидация выполняются на остальных, менее крупных конфигурациях.

Для стабилизации обучения целевые переменные переводятся в логарифмическое пространство:

$$\begin{aligned}\tilde{T}_i &= \log(1 + T_i), & \widetilde{M}_{\text{peak},i} &= \log(1 + M_{\text{peak},i}), & \tilde{E}_i &= \log(1 + E_i), \\ \hat{\tilde{T}}_i &= \log(1 + \hat{T}_i), & \hat{\widetilde{M}}_{\text{peak},i} &= \log(1 + \hat{M}_{\text{peak},i}), & \hat{\tilde{E}}_i &= \log(1 + \hat{E}_i).\end{aligned}$$

Такое преобразование уменьшает влияние выбросов и сглаживает различия в масштабе между различными конфигурациями.

В качестве базовой функции потерь используется функция Хьюбера

$$\ell_\delta(r) = \begin{cases} \frac{1}{2}r^2, & |r| \leq \delta, \\ \delta \left(|r| - \frac{\delta}{2} \right), & |r| > \delta, \end{cases}$$

где r — ошибка предсказания, а $\delta > 0$ — параметр перехода от квадратичного штрафа к линейному. В работе используется значение $\delta = 1$.

Тогда функции потерь для отдельных целевых компонент задаются следующим образом:

$$\begin{aligned}\ell_T(\hat{T}_i, T_i) &= \ell_\delta(\tilde{\hat{T}}_i - \tilde{T}_i), \\ \ell_M(\hat{M}_{\text{peak},i}, M_{\text{peak},i}) &= \ell_\delta(\hat{\tilde{M}}_{\text{peak},i} - \tilde{M}_{\text{peak},i}), \\ \ell_E(\hat{E}_i, E_i) &= \ell_\delta(\hat{\tilde{E}}_i - \tilde{E}_i).\end{aligned}$$

Общая функция потерь в многокритериальной постановке имеет вид

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \left(\alpha_T \ell_T(\hat{T}_i, T_i) + \alpha_M \ell_M(\hat{M}_{\text{peak},i}, M_{\text{peak},i}) + \alpha_E \ell_E(\hat{E}_i, E_i) \right),$$

где $\alpha_T, \alpha_M, \alpha_E \in \mathbb{R}$ — веса компонент функции потерь, удовлетворяющие условиям

$$\alpha_T \geq 0, \quad \alpha_M \geq 0, \quad \alpha_E \geq 0, \quad \alpha_T + \alpha_M + \alpha_E = 1.$$

В работе основной акцент делается на качестве предсказания времени инференса, поэтому используются веса

$$(\alpha_T, \alpha_M, \alpha_E) = (0.6, 0.25, 0.15).$$

Качество предсказательной модели оценивается по средней абсолютной процентной ошибке отдельно для каждой целевой компоненты:

$$\begin{aligned} \text{MAPE}_T &= \frac{100\%}{N} \sum_{i=1}^N \left| \frac{T_i - \hat{T}_i}{T_i} \right|, \\ \text{MAPE}_M &= \frac{100\%}{N} \sum_{i=1}^N \left| \frac{M_{\text{peak},i} - \hat{M}_{\text{peak},i}}{M_{\text{peak},i}} \right|, \\ \text{MAPE}_E &= \frac{100\%}{N} \sum_{i=1}^N \left| \frac{E_i - \hat{E}_i}{E_i} \right|. \end{aligned}$$

Основным критерием сравнения моделей служит точность предсказания времени инференса, то есть величина MAPE_T .

4.7 Вычислительная платформа

Все измерения целевых метрик инференса для сбора обучающей выборки и вычислительные эксперименты в работе проводились на фиксированной вычислительной платформе $\mathcal{G}$. В качестве такой платформы использовался один графический ускоритель GPU NVIDIA A100, установленный на узле суперкомпьютера “270 лет МГУ”.

Фиксация вычислительной платформы позволяет исключить влияние межплатформенных различий на измеряемые характеристики и рассматривать задачу как предсказание метрик инференса для заранее заданной аппаратной среды. Таким образом, построенные предсказательные модели ориентированы на оценку времени инференса, пикового использования GPU-памяти и энергопотребления именно для данной вычислительной платформы.

5 Программная реализация метода

В данном разделе описывается программная реализация предложенного метода предсказания метрик инференса BERT-подобных энкодерных трансформер-моделей. Реализация организована как набор взаимосвязанных программных модулей, охватывающих полный цикл работы с данными: формирование набора BERT-подобных моделей, построение обучающей выборки, обучение предсказательных моделей, оценка качества и сохранение экспериментальных артефактов.

5.1 Общая архитектура программной реализации

Программная реализация включает следующие основные компоненты:

- модуль формирования набора BERT-подобных моделей, задающий перечень готовых моделей из Hugging Face и пользовательских BERT-конфигураций;
- модуль построения обучающей выборки, формирующий экземпляры датасета на основе вычислительного графа, признаков и измеренных целевых метрик;
- модуль обучения неграфовых предсказательных моделей на табличных представлениях признаков;
- модуль обучения графовой нейронной сети на полном графовом представлении вычислений;
- модуль оценки качества, сохранения результатов и формирования служебных артефактов эксперимента.

Общая схема программной реализации приведена на рис. 3.

Центральным элементом всей реализации является модуль построения обучающей выборки. Именно он связывает между собой архитектурную конфигурацию модели, параметры запуска инференса, входные последовательности, используемые при трассировке и измерении, и измеренные характеристики выполнения на целевой вычислительной платформе. На выходе этого модуля формируется единый файловый формат данных, используемый далее как в неграфовой, так и в графовой ветви обучения.

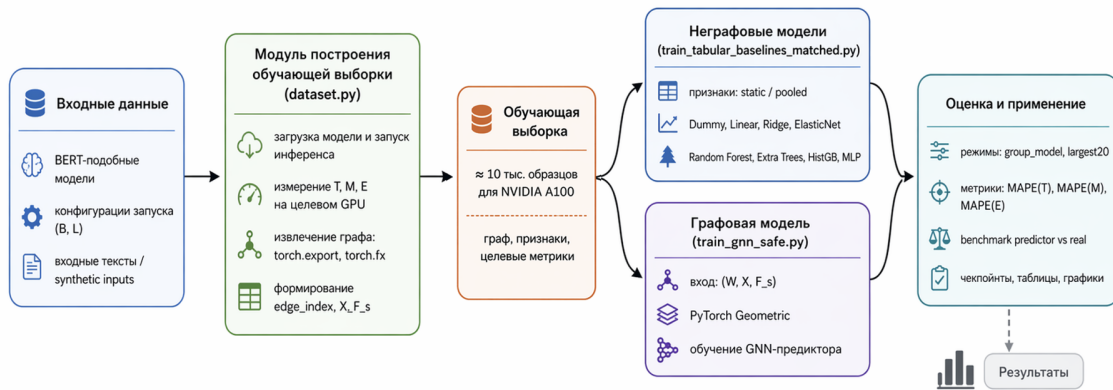


Рис. 3: Схема программной реализации метода предсказания метрик инференса BERT-подобных моделей

5.2 Модуль формирования набора моделей

Программная реализация поддерживает два источника моделей:

- готовые BERT-подобные encoder-only модели из библиотеки Hugging Face;
- пользовательские BERT-подобные модели, задаваемые программно с помощью функций-конструкторов.

Для пользовательских BERT-конфигураций используется отдельный модуль, формирующий сетку архитектур по числу слоёв, размеру скрытого состояния, числу голов внимания и размеру промежуточного слоя FFN.

Такое разделение позволяет одновременно использовать как реальные модели из распространённых семейств BERT-подобных энкодеров, так и синтетически заданные конфигурации, обеспечивающие более равномерное покрытие пространства архитектурных параметров.

5.3 Модуль построения обучающей выборки

Модуль построения обучающей выборки предназначен для автоматического формирования экземпляров датасета, соответствующих различным сочетаниям модели и конфигурации запуска инференса. В качестве входной информации используются:

- набор рассматриваемых BERT-подобных моделей;
- значения размера батча B и длины входной последовательности L , определяющие конфигурации запуска;
- данные, подаваемые на вход модели при трассировке и измерении;
- параметры запуска эксперимента и процедуры измерения метрик, включая используемое вычислительное устройство, число задействованных GPU, трассировочные параметры, число прогревочных и измерительных проходов.

В реализации предусмотрены два варианта входных данных, подаваемых на вход BERT-подобным моделям:

- реальные текстовые последовательности из корпуса CoLA (Corpus of Linguistic Acceptability);
- синтетически сгенерированные входные тензоры, согласованные с форматом входа рассматриваемой модели.

Для унификации интерфейса различных моделей используется специальная обёртка `EncoderOnlyWrapper`. Её назначение состоит в приведении прямого прохода различных BERT-подобных encoder-only моделей к единому формату вызова и единому типу выходных данных.

При инициализации обёртка анализирует сигнатуру метода `forward` конкретной модели и определяет, какие входные аргументы она поддерживает. В качестве унифицированного интерфейса используются аргументы `input_ids`, `attention_mask`, `token_type_ids`. Здесь `input_ids` задаёт последовательность идентификаторов токенов, подаваемых на вход модели; `attention_mask` определяет, какие позиции входной последовательности должны учитываться при вычислениях, а какие соответствуют служебному дополнению; `token_type_ids` задаёт идентификаторы сегментов входной последовательности и используется, например, при обработке пар предложений. Однако при фактическом вызове модели передаются только те из указанных аргументов, которые действительно предусмотрены её реализацией. Это позволяет использовать одну и ту же схему запуска для моделей, различающихся по набору входных параметров.

Отдельной задачей обёртки является нормализация формата выходных данных. В зависимости от используемой реализации модель может возвращать кортеж тензоров, объект с полем `last_hidden_state`, объект с полем `logits` или иной контейнер. Обёртка приводит эти варианты к единому виду, выделяя основной выходной тензор. Тем самым последующие этапы программной реализации, включая построение вычислительного графа и измерение характеристик инференса, могут работать с единым типом результата независимо от особенностей конкретной модели.

5.3.1 Построение графового представления

Для получения графового представления модели используется связка `torch.export` и `torch.fx`. Сначала строится статическое представление прямого прохода модели, то есть вычислительный граф с фиксированной структурой операций и связей между ними для заданного формата входа. Затем выполняется распространение информации о формах тензоров с использованием `ShapeProp` или, при необходимости, `FakeTensorProp`. Механизм `ShapeProp` выполняет проход по графу с распространением сведений о размерах промежуточных тензоров, тогда как `FakeTensorProp` решает аналогичную задачу с использованием специальных фиктивных тензоров без необходимости полноценного вычисления на реальных данных.

После этого из графа удаляются служебные тензорные атрибуты, а сами узлы типа `placeholder`, `get_attr` и `output` исключаются из итогового графового представления, используемого для построения датасета. Здесь узлы `placeholder` соответствуют входам графа, `get_attr` — обращениям к сохранённым параметрам и другим атрибутам модели, а `output` — выходу вычислительного графа. Такие узлы необходимы для внутреннего представления вычислений, однако не рассматриваются как содержательные операции инференса при формировании графовых признаков.

На основе построенного графа формируются:

- список рёбер W ;
- матрица локальных признаков узлов X ;
- вектор глобальных признаков F_s .

5.3.2 Измерение целевых метрик

Измерение целевых метрик организовано непосредственно в модуле построения выборки. Для каждой конфигурации выполняются этапы прогрева (`warmup`) и последующего измерения. Инференс выполняется в режиме `torch.inference_mode()`, а временные характеристики вычисляются по батчам с синхронизацией CUDA-устройств.

Энергопотребление GPU измеряется с использованием библиотеки NVML. В основном режиме для каждого задействованного устройства перед началом измерительной серии считывается значение накопленной энергии, после чего аналогичное считывание выполняется по завершении серии. Искомая величина энергопотребления определяется как разность между конечным и начальным значениями.

Если прямое чтение накопленной энергии недоступно, используется резервный режим оценки по мощности. В этом случае в процессе измерительных проходов для задействованного GPU периодически считывается значение текущей мощности, после чего вычисляется средняя мощность за время эксперимента. Оценка затраченной энергии определяется как произведение этой средней мощности на полное время измерительной серии.

Пиковое использование GPU-памяти определяется по значениям `torch.cuda.max_memory_allocated` для задействованного устройства. В качестве целевой метрики сохраняется максимальное из этих значений, поскольку именно оно характеризует наибольшую нагрузку на GPU и, следовательно, наиболее непосредственно связано с риском выхода за пределы доступной памяти.

5.3.3 Организация хранения выборки

Каждому экземпляру обучающей выборки соответствует отдельный файл формата `.npz`, содержащий графовое представление, признаки и целевые метрики. Дополнительно для всей серии экспериментов ведётся файл `meta.json`, в котором фиксируются:

- спецификация и имя модели;
- параметры запуска инференса;
- информация о вычислительном устройстве;
- характеристики построенного графа;

- способ измерения энергии;
- служебная информация о текущем запуске.

Реализация поддерживает режим продолжения ранее начатой генерации выборки (resume), при котором уже построенные конфигурации не пересчитываются повторно. Для повышения устойчивости длительных запусков в коде предусмотрены очистка памяти, журналирование использования памяти и атомарное обновление метаданных.

5.4 Модули обучения предсказательных моделей

После построения обучающей выборки данные используются в двух независимых ветвях: для обучения неграфовых и графовой моделей.

5.4.1 Модуль обучения неграфовых моделей

Модуль обучения неграфовых моделей работает с табличными представлениями признаков и поддерживает два режима:

- *static* — использование только глобальных признаков;
- *pooled* — использование глобальных признаков, дополненных агрегатами по матрице локальных признаков узлов.

В варианте *pooled* из матрицы X вычисляются статистики по узлам графа: среднее значение, стандартное отклонение и максимум по каждому признаку. После этого они конкатенируются с нормализованным вектором глобальных признаков.

Перед обучением применяется преобразование целевой переменной - $\log_{1p}$. Такое преобразование позволяет корректно обрабатывать нулевые значения и уменьшает различия в масштабе между малыми и большими значениями целевой переменной. Качество моделей оценивается на валидационной и тестовой выборках; дополнительно вычисляются метрики MAE, RMSE, R^2 и MAPE. Результаты по каждому сочетанию «режим, модель» сохраняются в отдельных json-файлах, а сводная информация агрегируется в итоговые файлы `results.json` и `results.tsv`.

5.4.2 Модуль обучения графовой модели

Для графовой ветви используется собственный класс датасета, загружающий экземпляры из файлов `.npz`, проверяющий корректность их структуры и формирующий объекты библиотеки PyTorch Geometric.

Архитектура графовой модели реализована в классе `PMGNSBert`. Графовая часть состоит из нескольких последовательно применяемых слоёв `SAGEConv`, каждый из которых сопровождается нелинейностью `ReLU` и слоем `Dropout`. После этого узловые представления агрегируются с помощью `global_mean_pool`, а полученный эмбединг графа объединяется с вектором глобальных признаков. Полносвязная часть модели содержит два линейных слоя с нелинейностями и регуляризацией, после чего формируется итоговый вектор предсказаний.

Обучение выполняется с использованием оптимизатора `Adam`. В качестве функции потерь используется взвешенная функция Хьюбера, применяемая к логарифмированным целевым переменным. Для управления процессом обучения реализована ранняя остановка по выбранной валидационной метрике. В ходе обучения сохраняются:

- сохранённое состояние параметров модели, соответствующее наилучшему значению выбранной валидационной метрики;
- история изменения метрик по эпохам;
- при необходимости — графики обучения и описание разбиения выборки.

5.5 Модуль разбиения данных и оценки качества

В обеих ветвях обучения используется единый механизм разбиения данных на обучающую, валидационную и тестовую части. В рамках вычислительных экспериментов используются следующие основные режимы:

- `random` — случайное разбиение, при котором каждый экземпляр рассматривается независимо;
- `group_model` — групповое разбиение по модели, при котором все экземпляры, соответствующие одной и той же модели, целиком относятся только к одному из подмножеств выборки;

- **largest20** — специальный режим, в котором тестовая выборка формируется из примерно 20% крупнейших конфигураций моделей, что позволяет оценивать способность предсказательной модели к экстраполяции на более крупные архитектуры. В скриптах это реализовано через выбор доли групп с наибольшим медианным значением числа параметров `total_params` и их полное перенесение в тестовую выборку.

5.6 Используемые программные библиотеки

- **PyTorch** — построение и обучение нейросетевых моделей, выполнение инференса и измерений;
- `torch.export` / `torch.fx` — получение и обработка вычислительного графа;
- **PyTorch Geometric** — реализация графовой нейронной сети;
- **scikit-learn** — реализация неграфовых предсказательных моделей;
- **Transformers** — загрузка готовых BERT-подобных моделей;
- **datasets** — загрузка корпуса CoLA;
- **NumPy** / **json** / **pathlib** — хранение и обработка данных и экспериментальных артефактов;
- **pynvml** — измерение характеристик энергопотребления и обращение к счётчикам GPU;
- **matplotlib** — сохранение графиков истории обучения графовой модели.

5.7 Особенности реализации

К наиболее важным особенностям программной реализации относятся:

- **Единый интерфейс для различных encoder-only BERT-подобных моделей.** За счёт специальной обёртки обеспечивается единообразная обработка как готовых моделей из библиотеки Hugging Face, так и пользовательских BERT-конфигураций, задаваемых программно;

- **Автоматическое построение графового представления.** Вычислительный граф извлекается программно, после чего на его основе формируются графовые структуры, локальные признаки узлов и агрегированные глобальные характеристики модели;
- **Поддержка различных источников входных данных.** Для построения выборки могут использоваться как реальные текстовые последовательности, так и синтетически сгенерированные входные тензоры, согласованные с форматом входа рассматриваемой модели;
- **Устойчивость длительных запусков.** В генераторе выборки предусмотрены режим возобновления ранее начатого эксперимента, атомарная запись метаданных, повторные попытки загрузки внешних ресурсов, очистка оперативной и видеопамяти, а также журналирование потребления вычислительных ресурсов;
- **Разделение этапов сбора данных и последующего обучения.** Наличие целевой вычислительной платформы является обязательным условием на этапе построения обучающей выборки, когда выполняются измерения времени инференса, использования памяти и энергопотребления. После формирования датасета обучение предсказательных моделей, валидация и тестирование выполняются по сохранённым экземплярам выборки и не требуют повторного доступа к той вычислительной платформе, на которой были собраны исходные измерения;
- **Поддержка многокритериального предсказания.** Во всех основных модулях сохранена возможность одновременной работы с несколькими целевыми компонентами, включая время инференса, пиковое использование GPU-памяти и энергопотребление;
- **Отсутствие жёсткой привязки к конкретной аппаратной платформе.** Несмотря на то что основная экспериментальная часть работы проводилась на GPU NVIDIA A100, программные скрипты не содержат жёстко зафиксированной зависимости от этой модели ускорителя. Выбор устройства осуществляется в общем виде через CPU/CUDA-механизм PyTorch, а число используемых GPU определяется динамически по доступным устройствам. Это позволяет применять

ту же программную реализацию и на других вычислительных платформах без модификации исходного кода; при необходимости для новой платформы может быть заново построена обучающая выборка и выполнено переобучение предикторов.

Таким образом, разработанная программная реализация представляет собой воспроизводимый и расширяемый программный конвейер, охватывающий все основные этапы решения задачи: формирование обучающей выборки, построение признаковых представлений, обучение предсказательных моделей, оценку качества и сохранение результатов вычислительных экспериментов.

6 Результаты экспериментов

В данном разделе приводятся результаты экспериментального исследования предсказательных моделей, обученных на построенной выборке для оценки метрик инференса BERT-подобных энкодерных трансформеров.

Качество моделей оценивалось по средней абсолютной процентной ошибке (MAPE) отдельно для каждой из трёх целевых метрик. Основным критерием выбора модели являлась ошибка по времени инференса T , поскольку именно эта метрика является ключевой при выборе эффективной конфигурации запуска.

6.1 Результаты в режиме *group_model*

В таблице 8 приведены лучшие результаты, полученные в режиме *group_model*, в котором оценивается способность предсказательных моделей переноситься на архитектурные конфигурации, полностью отсутствующие в обучающей выборке.

Таблица 8: Ошибки предсказания метрик в режиме *group_model*, %

Модель	Представление	MAPE(T), %	MAPE(M), %	MAPE(E), %
HistGB	pooled	5.44	4.95	8.13
GNN	graph	5.47	5.71	8.61
HistGB	static	5.65	6.05	8.18
ExtraTrees	pooled	5.99	4.69	8.81

Как следует из таблицы 8, в режиме *group_model* наилучшее качество по основной целевой метрике — времени инференса — достигается моделью HistGB, использующей представление *pooled*. Средняя абсолютная процентная ошибка в этом случае составляет 5.44%. Графовая нейронная сеть демонстрирует практически сопоставимый результат: значение MAPE(T) для неё равно 5.47%. Столь малая разница указывает на то, что явное графовое представление вычислений действительно содержит существенную информацию о структуре модели, релевантную для предсказания времени инференса.

По метрике пикового использования GPU-памяти наилучший результат показывает модель ExtraTrees на *pooled*-признаках, для которой MAPE(M) составляет 4.69%.

По метрике энергопотребления лучшей вновь оказывается модель HistGB на *pooled*-признаках с ошибкой 8.13%.

В целом результаты режима *group_model* показывают, что задача переноса на новые BERT-подобные модели решается с высокой точностью как при использовании табличных представлений, дополненных агрегированной информацией о вычислительном графе, так и при использовании явного графового представления. При этом представление *pooled* оказывается наиболее устойчивым для неграфовых моделей, а графовая нейронная сеть демонстрирует конкурентоспособное качество по основной метрике времени инференса.

6.2 Результаты в режиме *largest20*

В таблице 9 приведены лучшие результаты для режима *largest20*, соответствующего задаче экстраполяции на наиболее крупные архитектурные конфигурации, отсутствовавшие в обучающей выборке.

Таблица 9: Ошибки предсказания метрик в режиме *largest20*, %

Модель	Представление	MAPE(T), %	MAPE(M), %	MAPE(E), %
HistGB	static	8.68	32.95	9.95
HistGB	pooled	9.37	32.57	10.03
ExtraTrees	static	9.54	37.76	9.92
ExtraTrees	pooled	9.93	36.48	10.10
GNN	graph	13.85	59.10	30.74

Как следует из таблицы 9, в режиме *largest20* наилучшее качество по основной целевой метрике — времени инференса — достигается моделью HistGB на *static*-признаках. Значение MAPE(T) в этом случае составляет 8.68%. Использование представления *pooled* приводит к близкому, но несколько худшему результату: ошибка возрастает до 9.37%. Это указывает на то, что при экстраполяции на существенно более крупные архитектуры глобальные признаки модели и конфигурации запуска оказываются более устойчивыми, чем агрегированные характеристики локальных признаков узлов графа.

Наиболее сложной для предсказания в данном режиме оказывается метрика пиково-

го использования GPU-памяти. Даже для лучшей модели значение $\text{MAPE}(M)$ составляет 32.57%, что существенно выше соответствующих ошибок в режиме *group_model*. Такой результат объясняется тем, что пиковое потребление памяти определяется не только масштабом архитектуры и параметрами запуска, но и деталями структуры вычислительного графа, особенностями размещения промежуточных тензоров и спецификой работы механизма выделения памяти во фреймворке. По этой причине при выходе в область более крупных архитектур память оказывается значительно менее устойчивой к экстраполяции, чем время инференса и энергопотребление.

По метрике энергопотребления лучшей оказывается модель ExtraTrees на *static*-признаках с ошибкой 9.92%, однако различия между лучшими неграфовыми моделями по этой целевой компоненте остаются сравнительно небольшими. В то же время графовая нейронная сеть в режиме *largest20* демонстрирует заметно более низкое качество по всем трём метрикам: для неё $\text{MAPE}(T)$ достигает 13.85%, $\text{MAPE}(M)$ — 59.10%, а $\text{MAPE}(E)$ — 30.74%. Это позволяет заключить, что в условиях жёсткой экстраполяции на наиболее крупные архитектуры используемое графовое представление и объём доступных обучающих данных оказываются недостаточными для устойчивого переноса.

В целом результаты режима *largest20* показывают, что при переходе к наиболее крупным архитектурам наибольшую устойчивость демонстрируют неграфовые модели, использующие компактное глобальное описание архитектуры и конфигурации запуска. При этом задача предсказания времени инференса и энергопотребления остаётся решаемой с приемлемой точностью, тогда как предсказание пикового использования GPU-памяти существенно усложняется.

6.3 Сравнение режимов и представлений

Сопоставление результатов, полученных в режимах *group_model* и *largest20*, позволяет выявить несколько устойчивых закономерностей, характеризующих как свойства используемых признаков представлений, так и особенности обобщающей способности рассматриваемых предсказательных моделей.

Прежде всего, в режиме *group_model* графовая нейронная сеть демонстрирует качество, практически совпадающее с качеством лучшей неграфовой модели по основной целевой метрике — времени инференса. Значения $\text{MAPE}(T)$ для GNN и HistGB составляют соответственно 5.47% и 5.44%, то есть различаются лишь на 0.03 процент-

ного пункта. Такой результат свидетельствует о том, что явное графовое представление вычислений действительно содержит существенную информацию о структуре модели, значимую для предсказания её производительности.

В режиме *largest20*, напротив, наилучший результат по времени инференса достигается моделью HistGB на *static*-признаках, для которой $\text{MAPE}(T)$ составляет 8.68%. Это указывает на то, что при экстраполяции на наиболее крупные архитектуры более устойчивыми оказываются именно глобальные признаки модели и параметров запуска. В отличие от локальных или агрегированных структурных характеристик, такие признаки непосредственно отражают масштаб архитектуры, включая число слоёв, размер скрытого состояния, число голов внимания, размер промежуточного слоя FFN и параметры входной нагрузки.

Кроме того, представление *pooled* оказывается наиболее полезным в режиме *group_model*, где требуется перенос на новые модели сопоставимого масштаба. В этом случае агрегированная информация о локальных признаках узлов вычислительного графа позволяет точнее учитывать различия между архитектурами и тем самым повышать качество предсказания. В режиме *largest20* данное преимущество не сохраняется: использование *pooled*-признаков не приводит к улучшению по основной метрике времени инференса по сравнению с компактным *static*-описанием. Это позволяет заключить, что при выходе в область более крупных архитектур избыточная детализация признакового описания не всегда способствует повышению устойчивости модели и может уступать более компактному глобальному представлению.

6.4 Анализ вычислительной стоимости применения предиктора

Дополнительно была исследована вычислительная стоимость практического применения лучшей предсказательной модели — HistGB. В рамках данного анализа сопоставлялись два способа получения оценки времени инференса: прямое измерение на целевой вычислительной платформе и предсказание с использованием обученной модели. При этом необходимо различать два принципиально разных сценария применения предиктора:

- первичная оценка новой модели, включающая построение её графового представления;

- повторная оценка различных конфигураций запуска для модели, графовое представление которой уже было построено ранее.

Проведённый анализ показывает, что в сценарии первичной оценки основная вычислительная стоимость приходится на этап построения графового представления модели. Доля этого этапа составляет около 99.8% от общего времени выполнения всей процедуры оценки. По этой причине первая полная оценка новой модели оказывается более затратной по времени, чем единичное прямое измерение инференса на GPU.

Иная картина наблюдается в сценарии повторного использования уже построенного графа. В этом случае предсказание времени инференса для различных конфигураций запуска выполняется заметно быстрее прямого измерения. Полученные результаты показывают, что медианное ускорение составляет около 2.7–3.0 раза, а для наиболее тяжёлых конфигураций достигает 5–6 раз. Таким образом, практический выигрыш от использования предиктора возникает прежде всего в тех случаях, когда требуется многократно оценивать различные конфигурации запуска одной и той же модели.

Для количественного описания области применимости была рассмотрена точка безубыточности, то есть минимальное число конфигураций запуска, начиная с которого суммарное время работы предсказательной процедуры оценки становится меньше суммарного времени прямых измерений. Если для одной и той же модели графовое представление строится однократно, а затем выполняется оценка K конфигураций запуска, то суммарное время получения оценок в случае прямых измерений и в случае использования предсказательной модели задаётся выражениями

$$T_{\text{direct}}(K) = K \cdot \bar{t}_{\text{real}}, \quad T_{\text{pred}}(K) = t_{\text{graph}} + K \cdot \bar{t}_{\text{pred}},$$

где $\bar{t}_{\text{real}}$ — среднее время одного прямого измерения, t_{graph} — время построения графового представления, а $\bar{t}_{\text{pred}}$ — среднее время одной оценки с использованием предсказательной модели после построения графа.

Тогда точка безубыточности определяется выражением

$$K_{\text{break-even}} = \left\lceil \frac{t_{\text{graph}}}{\bar{t}_{\text{real}} - \bar{t}_{\text{pred}}} \right\rceil, \quad \bar{t}_{\text{real}} > \bar{t}_{\text{pred}}.$$

Например, для модели **roberta-large** полученная точка безубыточности составляет порядка 187–191 конфигураций запуска. Эти результаты показывают, что предложен-

ную процедуру оценки не следует рассматривать как замену единичному прямому измерению инференса модели. Однако метод оказывается практически целесообразным в сценариях массового перебора и оценки большого числа конфигураций запуска, когда затраты на однократное построение графового представления компенсируются ускорением последующих предсказаний.

Следует отметить, что сравнительно высокая точка безубыточности в рассматриваемых экспериментах во многом обусловлена спецификой самого класса исследуемых моделей. BERT-подобные энкодерные трансформеры относятся к числу сравнительно менее ресурсоёмких моделей по сравнению с современными большими языковыми моделями, вследствие чего стоимость прямого измерения их инференса на целевой вычислительной платформе остаётся относительно невысокой. В таких условиях выигрыш от замены прямых измерений предсказательной процедурой проявляется лишь при достаточно большом числе оцениваемых конфигураций запуска.

Можно ожидать, что для более тяжёлых моделей, прежде всего современных LLM-архитектур, ситуация будет иной: вследствие существенно более высокой стоимости одного прямого запуска потенциальный выигрыш от использования предиктора должен возрасти, а соответствующая точка безубыточности — снижаться. Таким образом, полученные в работе значения точки безубыточности следует интерпретировать с учётом того, что они характерны именно для класса BERT-подобных моделей и не могут напрямую переноситься на существенно более ресурсоёмкие архитектуры.

6.5 Выводы по результатам экспериментов

Проведённое экспериментальное исследование позволяет сделать следующие выводы.

Во-первых, в режиме *group_model*, соответствующем переносу на новые архитектурные конфигурации, наилучшее качество по основной целевой метрике — времени инференса — демонстрирует модель HistGB на *pooled*-признаках. При этом графовая нейронная сеть показывает практически сопоставимый результат, что подтверждает информативность явного графового представления вычислений для задачи предсказания метрик инференса.

Во-вторых, в режиме *largest20*, соответствующем экстраполяции на наиболее крупные архитектуры, наилучшей по времени инференса оказывается модель HistGB на *static*-признаках. Это указывает на то, что при выходе в область более крупных моделей

наиболее устойчивыми оказываются глобальные признаки архитектуры и конфигурации запуска, тогда как более детализированные структурные представления не всегда обеспечивают дополнительное улучшение качества.

В-третьих, среди рассматриваемых целевых метрик наиболее устойчиво предсказывается время инференса. Энергопотребление также оценивается с приемлемой точностью, тогда как наиболее сложной для предсказания оказывается метрика пикового использования GPU-памяти, особенно в условиях экстраполяции на крупные архитектуры.

В-четвёртых, практическая эффективность предсказательной процедуры существенно зависит от сценария её применения. Для первичной оценки новой модели основная вычислительная стоимость связана с построением графового представления, вследствие чего такой сценарий не даёт выигрыша по времени по сравнению с единичным прямым измерением. Однако при многократной оценке различных конфигураций запуска для одной и той же модели использование предиктора становится вычислительно оправданным и обеспечивает заметное ускорение.

Таким образом, результаты экспериментов показывают, что предложенное признаковое описание и построенные предсказательные модели позволяют с высокой точностью оценивать метрики инференса BERT-подобных энкодерных трансформеров на фиксированной вычислительной платформе. Полученные результаты также подтверждают, что предложенный подход представляет практический интерес в сценариях массовой оценки большого числа конфигураций запуска.

Заключение

Выпускная квалификационная работа посвящена задаче предсказания метрик инференса BERT-подобных энкодерных трансформерных моделей на фиксированной вычислительной платформе по описанию архитектуры модели и параметрам запуска. Актуальность рассматриваемой задачи определяется широким использованием BERT-подобных моделей в прикладных и промышленных системах, существенным разнообразием их архитектурных конфигураций, а также практической необходимостью выбора допустимых и эффективных режимов инференса без трудоёмкого перебора большого числа вариантов посредством прямых измерений на GPU.

В ходе работы были проанализированы основные группы существующих подходов к предсказанию метрик инференса нейросетевых моделей: аналитические методы, методы, основанные на профилировании, и методы, основанные на машинном обучении. Показано, что аналитические методы обладают высокой скоростью и интерпретируемостью, однако ограниченно учитывают особенности реального исполнения моделей; методы профилирования позволяют точнее отражать поведение целевой платформы, но требуют значительного объёма предварительных измерений. Наиболее перспективным направлением для рассматриваемой постановки являются методы машинного обучения, поскольку после этапа обучения они позволяют быстро оценивать новые конфигурации запуска без необходимости выполнения полного набора прямых измерений.

В работе разработан метод предсказания метрик инференса BERT-подобных моделей, основанный на структурном представлении модели в виде вычислительного графа операций и на использовании глобальных признаков, описывающих архитектуру модели и параметры запуска. Предложенное признаковое описание объединяет локальные характеристики узлов графа, агрегированные глобальные признаки и информацию о конфигурации инференса. Для реализации метода были разработаны программные средства, обеспечивающие автоматическое построение вычислительного графа, извлечение локальных и глобальных признаков, измерение целевых метрик инференса и формирование обучающей выборки в едином формате.

На основе готовых моделей из библиотеки Hugging Face и пользовательских BERT-конфигураций была собрана обучающая выборка объёмом порядка 10^4 экземпляров для GPU NVIDIA A100 узла суперкомпьютера “270 лет МГУ”. На построенной выборке были обучены и сопоставлены несколько классов предсказательных моделей, включая

линейные модели, ансамблевые методы на основе деревьев решений, градиентный бустинг, полносвязные нейронные сети и графовую нейронную сеть.

Проведённое экспериментальное исследование показало, что наилучшее качество по основной целевой метрике — времени инференса — обеспечивает модель HistGradientBoosting. В режиме переноса на новые модели (*group_model*) лучшим оказалось представление *pooled*: средняя абсолютная процентная ошибка по времени инференса составила 5.44%. При этом графовая нейронная сеть продемонстрировала практически сопоставимое качество (5.47%), что подтверждает информативность явного графового представления вычислений для задачи предсказания производительности. В режиме экстраполяции на наиболее крупные архитектуры (*largest20*) наиболее устойчивой оказалась модель HistGradientBoosting на *static*-признаках с ошибкой 8.68% по времени инференса. Показано также, что наиболее сложной для предсказания целевой метрикой является пиковое использование GPU-памяти, особенно в условиях экстраполяции на более крупные архитектуры.

Дополнительно был проведён анализ вычислительной стоимости применения предсказательной модели. Установлено, что при первичной оценке новой модели основная доля времени приходится на этап построения её графового представления. Вследствие этого предложенный подход не следует рассматривать как замену единичному прямому измерению новой модели. Вместе с тем при многократной оценке большого числа конфигураций запуска для одной и той же модели использование предиктора оказывается вычислительно оправданным и обеспечивает ускорение по сравнению с прямыми измерениями. Таким образом, предложенный метод представляет практический интерес прежде всего в сценариях массового перебора конфигураций запуска.

Следует отметить, что сравнительно умеренный выигрыш в рассматриваемых экспериментах во многом обусловлен спецификой самого класса исследуемых моделей. BERT-подобные энкодерные трансформеры относятся к числу сравнительно менее ресурсоёмких архитектур, поэтому стоимость одного прямого измерения их инференса на целевой вычислительной платформе остаётся относительно невысокой. Вследствие этого эффект от замены прямых измерений предсказательной процедурой проявляется лишь при достаточно большом числе оцениваемых конфигураций запуска. Для существенно более тяжёлых архитектур, прежде всего современных больших языковых моделей, можно ожидать более выраженного практического выигрыша, поскольку стои-

мость одного прямого запуска для них значительно выше. Это позволяет рассматривать разработанный подход как особенно перспективный для задач, связанных с массовой оценкой конфигураций инференса ресурсоёмких трансформерных моделей.

После этапа оффлайн-сбора обучающей выборки на целевой платформе обученный предиктор может использоваться без постоянного доступа к ней, выступая в качестве её программного суррогата для рассматриваемого класса моделей.

Таким образом, цель работы достигнута: разработан, программно реализован и экспериментально исследован метод предсказания метрик инференса BERT-подобных моделей по вычислительному графу и параметрам запуска на фиксированной вычислительной платформе. Полученные результаты подтверждают перспективность как графового представления модели, так и табличных признаковых описаний, дополненных структурной информацией, для задач прогнозирования производительности инференса.

Перспективы дальнейших исследований связаны с расширением класса рассматриваемых моделей, в частности на decoder-only и encoder-decoder архитектуры, с более детальным моделированием памяти и жизненного цикла тензоров, с исследованием применимости разработанного подхода к более ресурсоёмким моделям, включая современные LLM-архитектуры.

Результаты работы были представлены на научной конференции «Ломоносовские чтения – 2026» и опубликованы в материалах конференции [17].

Список литературы

- [1] Attention Is All You Need / Ashish Vaswani, Noam Shazeer, Niki Parmar et al. // Advances in Neural Information Processing Systems (NeurIPS 2017). — 2017. <https://arxiv.org/abs/1706.03762>.
- [2] *Qi, Hang*. Paleo: A Performance Model for Deep Neural Networks / Hang Qi, Evan R. Sparks, Ameet Talwalkar // International Conference on Learning Representations (ICLR). — 2017. <https://openreview.net/forum?id=SyVWJ85lg>.
- [3] Estimating GPU Memory Consumption of Deep Learning Models / Yanjie Gao, Yu Liu, Hongyu Zhang et al. // Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20). — 2020. <https://dl.acm.org/doi/10.1145/3368089.3417050>.
- [4] LLMem: Estimating GPU Memory Usage for Fine-Tuning Pre-Trained LLMs / Taeho Kim, Yanming Wang, Vatshank Chaturvedi et al. // Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI-24). — 2024. <https://arxiv.org/abs/2404.10933>.
- [5] *Patwari, Rajeev*. Forecasting LLM Inference Performance via Hardware-Agnostic Analytical Modeling. — 2025. <https://arxiv.org/abs/2508.00904>.
- [6] nn-Meter: Towards Accurate Latency Prediction of Deep-Learning Model Inference on Diverse Edge Devices / Li Lyna Zhang, Shihao Han, Junyan Wei et al. // Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services (MobiSys '21). — 2021. <https://air.tsinghua.edu.cn/pdf/nn-Meter-Towards-Accurate-Latency-Prediction-of-Deep-Learning-Model-Inference-on-Diverse-Edge-Devices.pdf>.
- [7] Habitat: A Runtime-Based Computational Performance Predictor for Deep Neural Network Training / Geoffrey X. Yu, Yubo Gao, Pavel Golikov, Gennady Pekhimenko // 2021 USENIX Annual Technical Conference (USENIX ATC 21). — 2021. <https://www.usenix.org/conference/atc21/presentation/yu>.
- [8] Execution Time Modeling for CNN Inference on Embedded GPUs / Nouredine Bouhali, Hocine Ouarnoughi, Smail Niar, Abdellah Ait El Cadi //

Proceedings of the 18th ACM International Conference on Computing Frontiers. — 2021. <https://dl.acm.org/doi/10.1145/3444950.3447284>.

- [9] Augur: Modeling the Resource Requirements of ConvNets on Mobile Devices / Zongqing Lu, Swati Rallapalli, Kevin Chan et al. // *IEEE Transactions on Mobile Computing*. — 2021. — Vol. 20, no. 2. — Pp. 352–365. <https://ieeexplore.ieee.org/document/8863962>.
- [10] A Learned Performance Model for Tensor Processing Units / Sam Kaufman, Phitchaya Phothilimthana, Yanqi Zhou et al. // *Proceedings of Machine Learning and Systems*. — Vol. 3. — 2021. — Pp. 387–400. <https://proceedings.mlsys.org/paper/2021/file/85d8ce590ad8981ca2c8286f79f59954-Paper.pdf>.
- [11] *Panner Selvam, Karthick*. DIPPM: A Deep Learning Inference Performance Predictive Model Using Graph Neural Networks / Karthick Panner Selvam, Mats Brorsson // *Euro-Par 2023: Parallel Processing*. — Springer Nature Switzerland, 2023. — Pp. 3–16. <https://arxiv.org/abs/2303.11733>.
- [12] *Lee, Seonho*. Forecasting GPU Performance for Deep Learning Training and Inference / Seonho Lee, Amar Phanishayee, Divya Mahajan // *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. — 2025. <https://arxiv.org/abs/2407.13853>.
- [13] LLMCompass: Enabling Efficient Hardware Design for Large Language Model Inference / Hengrui Zhang, August Ning, Rohan Prabhakar, David Wentzlaff // *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. — 2024. <https://arxiv.org/abs/2312.03134>.
- [14] ReaLLM: A Trace-Driven Framework for Rapid Simulation of Large-Scale LLM Inference / Huwan Peng, Scott Davidson, C.-J. Richard Shi, Michael Taylor // *2025 IEEE 36th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. — 2025. https://www.bsg.ai/papers/Peng_ReaLLM_ASAP_2025.pdf.

- [15] *Karfakis, George*. RAPID-LLM: Resilience-Aware Performance Analysis of Infrastructure for Distributed LLM Training and Inference. — 2025. <https://arxiv.org/abs/2512.19606>.
- [16] SweetSpot: An Analytical Model for Predicting Energy Efficiency of LLM Inference / Hiari Pizzini Cavagna, Andrea Proia, Giacomo Madella et al. // Proceedings of the 17th ACM/SPEC International Conference on Performance Engineering (ICPE '26). — 2026. <https://arxiv.org/abs/2602.05695>.
- [17] *Агаджанян, Г. М.* Прогнозирование времени инференса больших языковых BERT-подобных моделей с учетом характеристик целевой вычислительной платформы / Г. М. Агаджанян, В. Ю. Вахрушев, Н. Н. Попова // Научная конференция «Ломоносовские чтения». — МГУ имени М. В. Ломоносова, факультет ВМК, Россия: 2026. — Рр. 121–122. — 23 марта – 5 апреля 2026 г.